

Núcleo de Situação de Saúde

Arquitetura, Integração e Deployment

Zadoque Carneiro

16 de setembro de 2026

Sumário

1	Visão Geral e Princípios Arquiteturais	3
1.1	Objetivo da arquitetura	3
1.2	Princípios	3
2	Arquitetura do Sistema	4
2.1	Fluxo de consulta	4
2.2	Fluxo de atualização de dados	5
2.3	Responsabilidades	5
3	Organização dos Repositórios	5
3.1	Repositórios	6
3.2	Responsabilidades da equipe atual	6
3.3	Por que não unificar os repositórios	6
4	Docker e Ambiente de Integração	6
4.1	Containers	7
4.2	Comunicação entre containers	7
4.3	Compose de integração	8
4.4	Ambientes	8
5	Pipeline de Dados e Artefatos	8
5.1	Camadas de dados	9
5.2	Papel do DataFrame	9
5.3	Papel do Parquet	9
5.4	API de inspeção do pipeline	10
5.5	Validação antes da publicação	10

6	Contratos e Fluxos de Integração	10
6.1	Contrato frontend–Java	10
6.2	Contrato Python–PostgreSQL	11
6.3	Fluxo de atualização	11
6.4	Fluxo de consulta	11
6.5	Versionamento	12
7	Observabilidade e Debugging	12
7.1	Cadeia de diagnóstico	12
7.2	API de inspeção	13
7.3	Logs	13
7.4	Validação automatizada	13
8	Segurança e Configuração	14
8.1	Segredos	14
8.2	Banco de dados	14
8.3	Java	14
8.4	Python	14
8.5	Ambientes	15
9	Deployment e Evolução	15
9.1	Desenvolvimento local	15
9.2	Integração contínua	15
9.3	Versionamento das imagens	15
9.4	Evolução da equipe	15
9.5	Evoluções futuras	16

1 Visão Geral e Princípios Arquiteturais

O Núcleo de Situação de Saúde (NSS) é organizado como uma aplicação web apoiada por um backend Java/Spring Boot, um pipeline de dados em Python e um banco de dados PostgreSQL. O objetivo desta documentação é registrar as decisões que permitem que os três componentes evoluam de forma independente, mas permaneçam integrados como um único sistema.

1.1 Objetivo da arquitetura

A arquitetura separa duas responsabilidades que possuem características diferentes:

- **consulta da aplicação:** realizada pelo frontend por meio da API Java, com leitura dos dados consolidados no PostgreSQL;
- **ingestão e atualização dos dados:** realizada pelo pipeline Python, que obtém dados do SINAN por meio do PySUS, processa os dados e atualiza o PostgreSQL.

Assim, o Python não faz parte do caminho síncrono de uma consulta feita pelo usuário. O usuário consulta dados que já foram processados e persistidos.

Princípio de separação de responsabilidades

O frontend não acessa diretamente o PostgreSQL nem o pipeline Python. A API Java é o ponto de entrada da aplicação. O pipeline Python é responsável pela preparação e atualização dos dados.

1.2 Princípios

1. Cada componente possui uma responsabilidade técnica bem definida.
2. Os três componentes de aplicação permanecem em repositórios independentes.
3. O Docker fornece ambientes reproduzíveis para desenvolvimento e integração.
4. O PostgreSQL é o armazenamento compartilhado dos dados utilizados pela aplicação.
5. DataFrames e arquivos Parquet permanecem no pipeline para processamento, rastreabilidade, inspeção e debugging.
6. O pipeline deve permitir identificar em qual etapa uma inconsistência foi introduzida.
7. Contratos de dados e mudanças de schema devem ser tratados de forma versionada.
8. Segurança, configuração e credenciais não devem ser incorporadas ao código-fonte.

i Decisão arquitetural

A separação em três repositórios não implica necessariamente três aplicações isoladas. Os componentes podem ser desenvolvidos separadamente e integrados em um único ambiente por meio de Docker Compose, mantendo uma rede Docker compartilhada e containers distintos.

2 Arquitetura do Sistema

A arquitetura do NSS utiliza quatro componentes executados separadamente: frontend, backend Java/Spring Boot, pipeline Python e PostgreSQL.

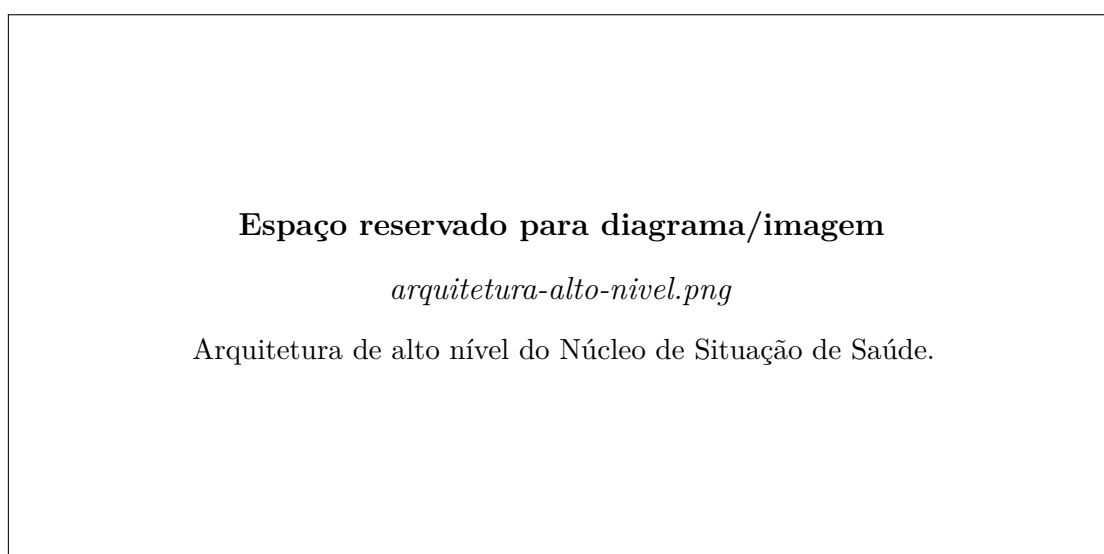


Figura 1: Arquitetura de alto nível do Núcleo de Situação de Saúde.

2.1 Fluxo de consulta

O fluxo normal de uma consulta é:

1. o usuário interage com o frontend;
2. o frontend envia uma requisição HTTP para a API Java;
3. o Java autentica e autoriza a operação, quando aplicável;
4. a camada de serviço aplica as regras de negócio;
5. o backend consulta o PostgreSQL;
6. o resultado é transformado em DTO e retornado ao frontend.

2.2 Fluxo de atualização de dados

A atualização dos dados ocorre separadamente do fluxo de consulta:

1. o pipeline Python obtém dados do SINAN por meio do PySUS;
2. os dados brutos são preservados em Bronze;
3. transformações e padronizações produzem Silver;
4. agregações necessárias à aplicação produzem Gold;
5. os resultados são validados;
6. os dados consolidados são persistidos no PostgreSQL.

Pipeline fora do caminho crítico de consulta

Uma consulta do usuário não deve depender de uma nova coleta ou processamento do SINAN. A aplicação consulta dados já persistidos e validados no PostgreSQL.

2.3 Responsabilidades

Componente	Responsabilidade
Frontend	Interface, visualizações, mapas, filtros e interação com o usuário.
Java/Spring Boot	API REST, autenticação, autorização, regras de negócio e acesso ao banco.
Python/Pipeline	Ingestão, transformação, agregação, validação e atualização dos dados.
PostgreSQL	Persistência compartilhada dos dados consolidados e das informações da aplicação.

3 Organização dos Repositórios

O projeto é mantido em três repositórios de aplicação. Esta decisão é intencional: permite que cada componente possua seu próprio ciclo de desenvolvimento, enquanto o repositório de deployment documenta e integra o sistema.

3.1 Repositórios

Componente	Repositório atual	Responsabilidade
Java	ArtursPereira/Site-Sala-de-Batida-principal-Saude-Spring Boot.	
Python	Zadoque/Nucleo-de-Situacao-de-Saude	Pipeline de dados e processamento SINAN/PySUS.
Deployment	Zadoque/nss-deployment	Documentação arquitetural e futura orquestração de integração.
Frontend	Repositório do frontend	Interface web e consumo da API Java.

3.2 Responsabilidades da equipe atual

Atualmente o projeto é desenvolvido por três estagiários:

- **Zadoque Carneiro:** arquitetura e documentação;
- **Artur Pereira:** Java e Spring Boot;
- **Gabriel Costa:** Python e pipeline de dados.

A organização em repositórios separados foi escolhida para continuar sendo adequada quando novos desenvolvedores e equipes forem incorporados ao projeto.

3.3 Por que não unificar os repositórios

Um monorepo seria tecnicamente possível, porém não é obrigatório para obter integração. Neste projeto, a separação permite que cada componente tenha seu próprio histórico, revisão, dependências e pipeline de CI/CD.

O custo adicional dessa escolha é compensado por contratos claros entre componentes e por um ambiente de integração centralizado.

i Repositório de deployment

O repositório `nss-deployment` não deve concentrar código de negócio. Ele funciona como ponto de integração operacional, documentação e, quando necessário, configuração de ambientes que combinam as imagens dos três componentes.

4 Docker e Ambiente de Integração

Os três repositórios permanecem independentes, mas podem ser executados em conjunto. A unidade de execução não precisa coincidir com a unidade de versionamento.

4.1 Containers

O ambiente integrado deverá possuir, conceitualmente, pelo menos:

- um container para o frontend;
- um container para o Java/Spring Boot;
- um container para o pipeline Python;
- um container para o PostgreSQL.

Cada componente de aplicação pode produzir sua própria imagem Docker. Não é necessário colocar Java, Python e frontend em uma única imagem.

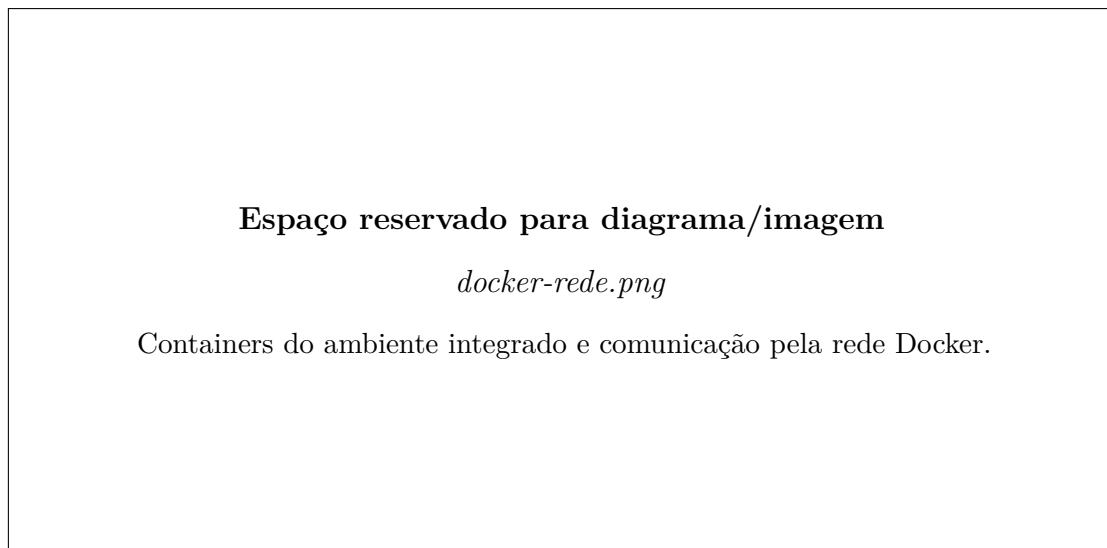


Figura 2: Containers do ambiente integrado e comunicação pela rede Docker.

4.2 Comunicação entre containers

Serviços executados na mesma rede Docker Compose podem se comunicar usando o nome do serviço como hostname. Assim, o backend Java pode acessar o PostgreSQL por um endereço conceitual como:

```
jdbc:postgresql://db:5432/sala_saude
```

O pipeline Python pode utilizar a mesma infraestrutura:

```
DB_HOST=db  
DB_PORT=5432  
DB_NAME=sala_saude
```

O endereço `localhost` não deve ser usado para alcançar outro container, pois dentro de um container `localhost` representa o próprio container.

4.3 Compose de integração

Recomenda-se que o repositório de deployment possua um Compose capaz de integrar as imagens dos três repositórios:

```
services:
  frontend:
    image: nss-frontend:dev

  java:
    image: nss-java:dev
    depends_on:
      - db

  pipeline:
    image: nss-pipeline:dev
    depends_on:
      - db

  db:
    image: postgres:16-alpine
```

Durante desenvolvimento, o Compose também pode usar `build.context` para construir as imagens a partir dos diretórios locais dos três repositórios.

4.4 Ambientes

A mesma separação pode ser utilizada em diferentes ambientes:

- **desenvolvimento:** imagens construídas localmente;
- **integração:** versões coordenadas dos três componentes;
- **produção:** imagens versionadas publicadas em um registry.

Isolamento das imagens

Cada componente deve possuir sua própria imagem Docker. A integração ocorre no nível de rede e orquestração, não pela fusão dos componentes em uma imagem única.

5 Pipeline de Dados e Artefatos

O pipeline Python é responsável por transformar os dados externos em dados confiáveis para a aplicação. O processamento utiliza DataFrames e preserva arquivos Parquet como artefatos intermediários.

5.1 Camadas de dados

A organização conceitual é:



- **Bronze:** preserva os dados de entrada e informações da ingestão;
- **Silver:** concentra limpeza, padronização e transformação;
- **Gold:** contém dados consolidados e agregações adequadas às consultas do sistema.

5.2 Papel do DataFrame

O DataFrame é uma estrutura de processamento do pipeline. Ele facilita transformações, validações, agrupamentos e inspeção durante a execução.

5.3 Papel do Parquet

Os arquivos Parquet são mantidos como artefatos persistentes do pipeline. Eles são úteis para:

- inspeção manual dos resultados;
- comparação entre etapas;
- reprodução de uma transformação sem coletar novamente os dados externos;
- investigação de falhas;
- auditoria técnica da evolução dos dados.

i Parquet não substitui o PostgreSQL

O Parquet continua sendo parte importante do pipeline, mas não é o armazenamento principal utilizado pelo frontend. O PostgreSQL funciona como fonte persistida dos dados consolidados para a aplicação Java.

5.4 API de inspeção do pipeline

O componente Python pode disponibilizar uma API de inspeção para desenvolvimento, validação e debugging. Essa API não deve ser confundida com a API oficial da aplicação.

Seu objetivo é permitir comparar diretamente o resultado do pipeline com o que posteriormente será armazenado no PostgreSQL e retornado pelo Java.

5.5 Validação antes da publicação

Antes de considerar uma execução concluída, o pipeline deve validar propriedades importantes dos dados. Exemplos:

- número de registros por etapa;
- presença das colunas esperadas;
- valores obrigatórios;
- totais por doença, ano e unidade geográfica;
- consistência entre Gold e dados persistidos no PostgreSQL.

Uma falha de validação deve ser registrada e impedir a publicação de dados inconsistentes, quando a regra de validação for crítica.

6 Contratos e Fluxos de Integração

A existência de três repositórios exige contratos claros. O objetivo é permitir que cada componente evolua sem criar dependências implícitas entre implementações internas.

6.1 Contrato frontend–Java

O frontend consome exclusivamente a API pública do Java. O contrato deve definir:

- endpoints;
- parâmetros e filtros;

- formato das respostas;
- códigos HTTP;
- mensagens de erro;
- paginação, quando aplicável;
- requisitos de autenticação e autorização.

6.2 Contrato Python–PostgreSQL

O pipeline e o Java compartilham o PostgreSQL. O schema do banco funciona, portanto, como um contrato de integração interno.

As mudanças de schema devem ser versionadas por migrations e acompanhadas de análise de compatibilidade com o backend Java.

Compatibilidade de schema

Uma alteração no PostgreSQL não deve ser publicada apenas porque o pipeline consegue executá-la. Deve-se verificar se as consultas e mapeamentos utilizados pelo Java continuam válidos.

6.3 Fluxo de atualização

```
sequenceDiagram
    participant S as SINAN
    participant P as Python Pipeline
    participant D as PostgreSQL

    P->>S: Obter dados
    S-->>P: Dados SINAN
    P->>P: Bronze / Silver / Gold
    P->>P: Validar dados
    P->>D: Atualizar dados consolidados
```

6.4 Fluxo de consulta

```
sequenceDiagram
    participant U as Usuário
    participant F as Frontend
    participant J as Java
    participant D as PostgreSQL
```

```
U->>F: Interage com a aplicação
F->>J: Requisição HTTP
J->>J: Autenticação / autorização
J->>D: Consulta
D-->>J: Dados
J-->>F: DTO / JSON
F-->>U: Visualização
```

6.5 Versionamento

Recomenda-se que contratos importantes sejam documentados e, quando possível, verificados automaticamente. A API Java pode utilizar OpenAPI para manter uma especificação formal de seus endpoints.

7 Observabilidade e Debugging

A arquitetura foi desenhada para permitir localizar uma inconsistência sem depender exclusivamente da inspeção manual do banco de dados.

7.1 Cadeia de diagnóstico

Quando um valor exibido no frontend parecer incorreto, a investigação deve seguir a cadeia:

1. verificar a resposta da API Java;
2. comparar com a consulta equivalente no PostgreSQL;
3. comparar o PostgreSQL com o Gold produzido pelo pipeline;
4. comparar o Gold com o Silver;
5. comparar o Silver com o Bronze;
6. verificar a origem SINAN/PySUS quando necessário.

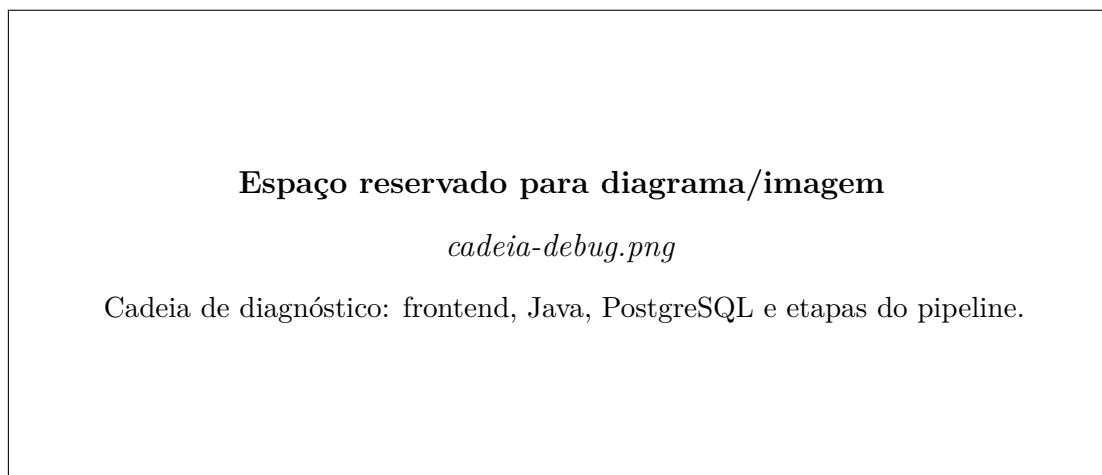


Figura 3: Cadeia de diagnóstico: frontend, Java, PostgreSQL e etapas do pipeline.

7.2 API de inspeção

A API de inspeção do Python pode disponibilizar consultas sobre os artefatos Gold. Ela deve ser tratada como ferramenta operacional e de desenvolvimento, e não como o contrato público da aplicação.

Isso permite uma comparação rápida:

Gold Python ↔ PostgreSQL ↔ Java API ↔ Frontend

7.3 Logs

Os três componentes devem produzir logs suficientes para identificar uma execução sem registrar dados sensíveis desnecessariamente. O pipeline deve registrar, no mínimo, identificadores de execução, origem, período processado, quantidade de registros e resultado das validações.

O Java deve registrar requisições relevantes, falhas de integração e erros de aplicação sem registrar credenciais ou informações sensíveis.

7.4 Validação automatizada

Sempre que possível, a comparação entre etapas deve ser automatizada. Exemplos de invariantes:

- schema esperado;
- quantidade mínima ou esperada de registros;
- ausência de chaves geográficas inválidas;
- consistência de totais;

- equivalência entre dados publicados no PostgreSQL e artefatos Gold.

Debug por camadas

A existência de um valor incorreto no frontend não deve levar imediatamente à conclusão de que o frontend está errado. Cada camada deve possuir meios próprios de inspeção para reduzir sistematicamente o espaço de busca do problema.

8 Segurança e Configuração

A segurança da aplicação é responsabilidade do backend Java. O frontend não deve possuir credenciais de banco de dados e não deve acessar diretamente serviços internos.

8.1 Segredos

Credenciais, chaves e tokens devem ser fornecidos por variáveis de ambiente, secrets do ambiente de execução ou mecanismo equivalente. Valores reais não devem ser versionados nos repositórios.

8.2 Banco de dados

O PostgreSQL deve ficar acessível diretamente apenas aos serviços que precisam dele. Em um ambiente integrado, o banco pode estar na rede interna do Compose sem necessidade de publicar sua porta para a máquina host.

8.3 Java

O backend deve aplicar autenticação e autorização antes de executar operações protegidas. Senhas devem ser armazenadas utilizando mecanismos de hashing apropriados, nunca em texto puro.

8.4 Python

A API de inspeção do pipeline deve receber proteção adequada quando exposta fora do ambiente local. Endpoints de debugging não devem expor informações desnecessárias em produção.

Princípio de menor exposição

Um serviço deve expor apenas as portas e endpoints necessários ao seu papel. PostgreSQL e endpoints internos não devem ser tratados como APIs públicas.

8.5 Ambientes

As configurações devem ser separadas por ambiente. O repositório pode manter exemplos, como `.env.example`, mas não valores secretos reais.

9 Deployment e Evolução

A estratégia de deployment deve preservar a independência dos três repositórios e, ao mesmo tempo, permitir a execução integrada do sistema.

9.1 Desenvolvimento local

Durante o desenvolvimento, as imagens podem ser construídas localmente a partir dos três repositórios. Um Compose de integração deve conectar frontend, Java, pipeline e PostgreSQL em uma rede comum.

9.2 Integração contínua

Em uma evolução futura, cada repositório poderá possuir seu próprio pipeline de CI/CD:

```
commit
|
v
CI -> testes -> build -> imagem Docker -> registry
```

O repositório de deployment poderá selecionar versões específicas das imagens para cada ambiente.

9.3 Versionamento das imagens

É recomendável utilizar tags imutáveis ou identificáveis, como versões semânticas ou SHA do commit. Um ambiente integrado pode então combinar, por exemplo, versões diferentes do frontend, Java e pipeline de maneira explícita.

9.4 Evolução da equipe

A separação por repositórios permite que equipes futuras assumam componentes diferentes. Para evitar que a separação gere acoplamento descontrolado, devem ser mantidos:

- contratos documentados;
- migrations versionadas;

- testes de integração;
- convenções de versionamento;
- documentação centralizada neste repositório.

9.5 Evoluções futuras

A arquitetura pode evoluir gradualmente para mecanismos como registry privado, CI/CD, observabilidade centralizada, jobs agendados e ambientes de staging. Essas ferramentas devem ser adotadas quando houver necessidade operacional real, evitando complexidade prematura.

i Princípio de evolução incremental

A arquitetura atual deve resolver o problema presente sem impedir crescimento futuro. Novas ferramentas e serviços devem ser introduzidos quando reduzirem um problema concreto de operação, manutenção ou escala.